

Usecase - Basic GRanges and AnnotationHub

Kasper D. Hansen

- [Dependencies](#)
- [Corrections](#)
- [Overview](#)
- [Getting the data](#)
- [SessionInfo](#)

Dependencies

This document has the following dependencies:

```
library(GenomicRanges)
library(rtracklayer)
library(AnnotationHub)
```

Use the following commands to install these packages in R.

```
source("http://www.bioconductor.org/biocLite.R")
biocLite(c("GenomicRanges", "rtracklayer", "AnnotationHub"))
```

Corrections

Improvements and corrections to this document can be submitted on its [GitHub](#) in its [repository](#).

Overview

We are going to use [AnnotationHub](#) and [GenomicRanges](#) to access ENCODE data on the H3K4me3 histone modification in a specific cell line. This histone modification is believed to mark active promoters, and we are going to attempt to verify this statement. This involves

1. Getting the ENCODE histone data using [AnnotationHub](#).
2. Getting promoters using [AnnotationHub](#).
3. Comparing the histone data and promoters using `findoverlaps` in [GenomicRanges](#).

Getting the data

First we use [AnnotationHub](#) to get data on homo sapiens.

```
ah <- AnnotationHub()  
ah <- subset(ah, species == "Homo sapiens")
```

Next we search for two keywords: **H3K4me3** and **Gm12878** which is the name of the cell line we are interested in.

```
qhs <- query(ah, "H3K4me3")  
qhs <- query(qhs, "Gm12878")
```

(Note: I like to keep my full annotation hub around, so I can re-do my query with a different search term in case I end up with no hits. This is why I start assigning output to the qhs object and not ah).

Lets have a look

```
qhs
```

```
## AnnotationHub with 11 records
## # snapshotDate(): 2015-08-17
## # $dataProvider: BroadInstitute, UCSC
## # $species: Homo sapiens
## # $rdaclass: GRanges, BigwigFile
## # additional mcols(): taxonomyid, genome, description, tags,
## #   sourceurl, sourcetype
## # retrieve records with, e.g., 'object[["AH23256"]]'
##
##           title
## AH23256 | wgEncodeBroadHistoneGm12878H3k4me3StdPk.broadPeak.gz
## AH27075 | wgEncodeUWHistoneGm12878H3k4me3StdHotspotsRep1.broadPeak.
## AH27076 | wgEncodeUWHistoneGm12878H3k4me3StdHotspotsRep2.broadPeak.
## AH27077 | wgEncodeUWHistoneGm12878H3k4me3StdPkRep1.narrowPeak.gz
## AH27078 | wgEncodeUWHistoneGm12878H3k4me3StdPkRep2.narrowPeak.gz
## ...     ...
## AH30747 | E116-H3K4me3.narrowPeak.gz
## AH31690 | E116-H3K4me3.gappedPeak.gz
## AH32869 | E116-H3K4me3.fc.signal.bigwig
## AH33901 | E116-H3K4me3.pval.signal.bigwig
## AH40294 | E116-H3K4me3.imputed.pval.signal.bigwig
```

Note how some of these hits don't contain **Gm12878** in their title. This is a useful illustration of how query searches over multiple fields.

Lets have a closer look at this

```
qhs$title
```

```
## [1] "wgEncodeBroadHistoneGm12878H3k4me3StdPk.broadPeak.gz"
## [2] "wgEncodeUWHistoneGm12878H3k4me3StdHotspotsRep1.broadPeak.gz"
## [3] "wgEncodeUWHistoneGm12878H3k4me3StdHotspotsRep2.broadPeak.gz"
## [4] "wgEncodeUWHistoneGm12878H3k4me3StdPkRep1.narrowPeak.gz"
## [5] "wgEncodeUWHistoneGm12878H3k4me3StdPkRep2.narrowPeak.gz"
## [6] "E116-H3K4me3.broadPeak.gz"
## [7] "E116-H3K4me3.narrowPeak.gz"
## [8] "E116-H3K4me3.gappedPeak.gz"
## [9] "E116-H3K4me3.fc.signal.bigwig"
## [10] "E116-H3K4me3.pval.signal.bigwig"
## [11] "E116-H3K4me3.imputed.pval.signal.bigwig"
```

```
qhs$dataprovder
```

```
## [1] "UCSC"          "UCSC"          "UCSC"          "UCSC"
## [5] "UCSC"          "BroadInstitute" "BroadInstitute" "BroadInstitu
## [9] "BroadInstitute" "BroadInstitute" "BroadInstitute"
```

This result is a great illustration of the mess of public data. It turns out that **E116** is a Roadmap Epigenetics code for the **Gm12878** cell line. The first 5 hits are from ENCODE, hosted at UCSC and the last 6 hits are from Roadmap Epigenomics hosted at the Broad Institute. The Roadmap data is different representation (and peaks) from the same underlying data. For the ENCODE data, two different centers did the same experiment in the same cell line (Broad, hit 1) and (Uw, hit 2-5), where Uw exposed data on two replicates (whatever that means). These two experiments seems to be analyzed using different algorithms. It is even possible that the Roadmap data is from the same raw data but just analyzed using different algorithms.

Lets take a look at the narrowPeak data:

```
gr1 <- subset(qhs, title == "wgEncodeUwHistoneGm12878H3k4me3StdPkRep1.na
gr1
```

```
## GRanges object with 74470 ranges and 6 metadata columns:
##           seqnames           ranges strand |           name
##           <Rle>             <IRanges> <Rle> | <character> <num
##      [1]      chr1      [713301, 713450]      * |      <NA>
##      [2]      chr1      [713501, 713650]      * |      <NA>
##      [3]      chr1      [713881, 714030]      * |      <NA>
##      [4]      chr1      [714181, 714330]      * |      <NA>
##      [5]      chr1      [714481, 714630]      * |      <NA>
##      ...      ...      ...      ...      ...      ...
## [74466]      chrX [154492741, 154492890]      * |      <NA>
## [74467]      chrX [154493401, 154493550]      * |      <NA>
## [74468]      chrX [154560441, 154560590]      * |      <NA>
## [74469]      chrX [154562121, 154562270]      * |      <NA>
## [74470]      chrX [154842061, 154842210]      * |      <NA>
##           signalValue    pValue    qValue    peak
##           <numeric> <numeric> <numeric> <numeric>
##      [1]           91  112.7680         -1         -1
##      [2]           25   26.7181         -1         -1
##      [3]           81   77.4798         -1         -1
##      [4]           32  106.5650         -1         -1
##      [5]          122  153.8320         -1         -1
##      ...      ...      ...      ...      ...
## [74466]           43   52.20930         -1         -1
## [74467]          122  203.16800         -1         -1
## [74468]            8    4.49236         -1         -1
## [74469]            8    4.41978         -1         -1
## [74470]          125  170.20100         -1         -1
## -----
## seqinfo: 23 sequences from an unspecified genome; no seqlengths
```

```
gr2 <- subset(qhs, title == "E116-H3K4me3.narrowPeak.gz")[[1]]
gr2
```

```
## GRanges object with 76188 ranges and 6 metadata columns:
##           seqnames           ranges strand |           name
##           <Rle>             <IRanges> <Rle> | <character> <num
##      [1]      chr9 [123553464, 123557122] * |      Rank_1
##      [2]      chr3 [ 53196213,  53197995] * |      Rank_2
##      [3]     chr18 [  9137534,   9142676] * |      Rank_3
##      [4]     chr11 [ 75110593,  75111943] * |      Rank_4
##      [5]     chr13 [ 41343776,  41345943] * |      Rank_5
##      ...      ...      ...      ...      ...
## [76184]     chr8 [ 98881411,  98881613] * | Rank_76184
## [76185]     chr9 [ 26812360,  26812533] * | Rank_76185
## [76186]     chrX [ 49019816,  49020016] * | Rank_76186
## [76187]     chrX [ 55932872,  55933045] * | Rank_76187
## [76188]     chr5 [118322235, 118322408] * | Rank_76188
##           signalValue    pValue    qValue    peak
##           <numeric> <numeric> <numeric> <numeric>
##      [1]      23.09216    62.34132    52.85911    1736
##      [2]      24.91976    62.22835    52.85911     439
##      [3]      22.48938    58.39180    50.67302     442
##      [4]      22.47056    54.84914    47.77176     752
##      [5]      20.82804    54.31837    47.29083     909
##      ...      ...      ...      ...      ...
## [76184]      2.63471     2.02539     0.27458      94
## [76185]      2.63471     2.02539     0.27458      32
## [76186]      2.63471     2.02539     0.27458      82
## [76187]      2.63471     2.02539     0.27458      41
## [76188]      2.62206     2.00970     0.26790      52
## -----
## seqinfo: 23 sequences from an unspecified genome; no seqlengths
```

(Note: I use this code, where I use title to refer to the different resources, to make this script more robust over time).

This gives us two GRanges objects. Let us look at the distribution of peak widths:

```
summary(width(gr1))
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    150.0   150.0   150.0   150.3   150.0   410.0
```

```
table(width(gr1))
```

```
##  
##    150    210    230    250    270    290    390    410  
## 74313      1     13     24     37     77      4      1
```

```
summary(width(gr2))
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.  
##      174     243     387     662     770    12340
```

Turns out that almost all peaks in gr1 have a width of 150bp, whereas gr2 is much more variable. This is likely a product of the data processing algorithm; it can be very hard to figure out the details of this.

At this time one can spend a lot of time thinking about which datasets is best. We will avoid this (important) discussion; since we referred to ENCODE data above (in the Overview section), we will stick with gr1.

Now we need to get some promoter coordinates. There are multiple ways to do this in Bioconductor, but here I will do a quick lookup for RefSeq in my annotation hub. RefSeq is a highly curated (aka conservative) collection of genes.

Lets get started

```
qhs <- query(ah, "RefSeq")  
qhs
```

```
## AnnotationHub with 8 records
## # snapshotDate(): 2015-08-17
## # $dataprovder: UCSC
## # $species: Homo sapiens
## # $rdaclass: GRanges
## # additional mcols(): taxonomyid, genome, description, tags,
## #   sourceurl, sourcetype
## # retrieve records with, e.g., 'object[["AH5040"]]'
##
##           title
## AH5040 | RefSeq Genes
## AH5041 | Other RefSeq
## AH5155 | RefSeq Genes
## AH5156 | Other RefSeq
## AH5306 | RefSeq Genes
## AH5307 | Other RefSeq
## AH5431 | RefSeq Genes
## AH5432 | Other RefSeq
```

So this gives us multiple datasets, all with very similar names. We probably need the thing called RefSeq Genes and not Other RefSeq but why are there multiple resources with the same name?

Turns out the answer makes sense:

```
qhs$genome
```

```
## [1] "hg19" "hg19" "hg18" "hg18" "hg17" "hg17" "hg16" "hg16"
```

This looks like the same resources, but in different genome builds. We have

```
genome(gr1)
```

```
## chr1 chr10 chr11 chr12 chr13 chr14 chr15 chr16 chr17 chr18 chr19 ch
##    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA
## chr20 chr21 chr22  chr3  chr4  chr5  chr6  chr7  chr8  chr9  chrX
##    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA
```

so we know which one to get:


```
refseq <- qhs[qhs$genome == "hg19" & qhs$title == "RefSeq Genes"]
refseq
```

```
## AnnotationHub with 1 record
## # snapshotDate(): 2015-08-17
## # names(): AH5040
## # $dataprovder: UCSC
## # $species: Homo sapiens
## # $rdataclass: GRanges
## # $title: RefSeq Genes
## # $description: GRanges object from UCSC track 'RefSeq Genes'
## # $taxonomyid: 9606
## # $genome: hg19
## # $sourcetype: UCSC track
## # $sourceurl: rtracklayer://hgdownload.cse.ucsc.edu/goldenpath/hg19/d
## # $sourcelastmodifieddate: NA
## # $sourcesize: NA
## # $tags: refGene, UCSC, track, Gene, Transcript, Annotation
## # retrieve record with 'object[["AH5040"]]'
```



```
refseq <- refseq[[1]] ## Downloads
```

Lets have a look

```
refseq
```

```
## UCSC track 'refGene'
## UCSCData object with 50066 ranges and 5 metadata columns:
##          seqnames          ranges strand |
##          <Rle>             <IRanges> <Rle> |
##      [1]          chr1 [66999825, 67210768]      + |
##      [2]          chr1 [ 8378145,  8404227]      + |
##      [3]          chr1 [48998527, 50489626]      - |
##      [4]          chr1 [16767167, 16786584]      + |
##      [5]          chr1 [16767167, 16786584]      + |
##      ...
## [50062] chr19_gl000209_random      [ 57209,  68123]      + |
## [50063] chr19_gl000209_random      [ 46646,  68123]      + |
## [50064] chr19_gl000209_random      [ 98135, 112667]      + |
## [50065] chr19_gl000209_random      [ 70071,  84658]      + |
## [50066] chr19_gl000209_random      [131433, 145745]      + |
##          name          score          itemRgb          thick
##          <character> <numeric> <character>          <IRanges>
##      [1]      NM_032291          0          <NA> [67000042, 67208778]
##      [2] NM_001080397          0          <NA> [ 8378169,  8404073]
##      [3]      NM_032785          0          <NA> [48999845, 50489468]
##      [4] NM_001145277          0          <NA> [16767257, 16785491]
##      [5] NM_001145278          0          <NA> [16767257, 16785385]
##      ...
## [50062]      NM_002255          0          <NA> [ 57249,  67717]
## [50063] NM_001258383          0          <NA> [ 57132,  67717]
## [50064]      NM_012313          0          <NA> [ 98146, 112480]
## [50065] NM_001083539          0          <NA> [ 70108,  83979]
## [50066]      NM_012312          0          <NA> [131468, 145120]
##          blocks
##          <IRangesList>
##      [1] [ 1, 227] [91706, 91769] [98929, 98953] ...
##      [2] [ 1, 102] [6222, 6642] [7214, 7306] ...
##      [3] [ 1, 1439] [2036, 2062] [6788, 6884] ...
##      [4] [ 1, 182] [2961, 3061] [7199, 7303] ...
##      [5] [ 1, 104] [2961, 3061] [7199, 7303] ...
##      ...
## [50062] [ 1, 80] [ 280, 315] [1182, 1466] ...
## [50063] [ 1, 86] [10414, 10643] [10843, 10878] ...
## [50064] [ 1, 46] [1523, 1557] [4002, 4301] ...
## [50065] [ 1, 71] [1071, 1106] [1851, 2135] ...
## [50066] [ 1, 69] [ 862, 897] [3334, 3633] ...
## -----
## seqinfo: 93 sequences from hg19 genome
```

Let us look at the number of isoforms per gene name:

```
table(table(refseq$name))
```

```
##  
##      1      2      3      4      5      6      7      8      9     10     12  
## 45178   568   102   27    74    52   171   129   13    19     1  
##      19  
##       5
```



(the `table(table())` construction may seem weird at first, but it's a great way to get a quick tabular summary of occurrences with the same name). This shows that almost all genes have a single transcript, which reflects how conservative RefSeq is.

Notice that each transcript is a separate range. Therefore, each transcript is represented as the “outer” coordinates of the gene; the ranges do not exclude introns. Because we got this from UCSC I happen to know that the `blocks` metadata really contains the coordinates of the different exons. In a later example we will introduce a gene representation where we keep track of exons, introns and transcripts from the same gene, through something known as a `TxDb` (transcript database) object.

For now, we just need the promoters, so we don't really care about introns. We need to keep track of which strand each transcript is on, to get the transcription start site.

Or we could just use the convenience function `promoters()`:

```
promoters <- promoters(refseq)  
table(width(promoters))
```

```
##  
## 2200  
## 50066
```

```
args(promoters)
```

```
## NULL
```

There are many definitions of promoters based on transcription start site. The default in this function is to use 2kb upstream and 200bp downstream of the start site. Let's keep this.

Now we compute which promoters have a H3K4me3 peak in them:

```
ov <- findOverlaps(promoters, gr1)
ov
```

```
## Hits object with 46022 hits and 0 metadata columns:
##           queryHits subjectHits
##           <integer>  <integer>
##      [1]           2           387
##      [2]           3          2523
##      [3]           4           774
##      [4]           5           774
##      [5]           6          1114
##      ...           ...           ...
## [46018]          47457          47491
## [46019]          47459          47493
## [46020]          47459          47494
## [46021]          47460          47493
## [46022]          47460          47494
## -----
## queryLength: 50066
## subjectLength: 74470
```

Let us compute how many percent of the peaks are in a promoter

```
length(unique(queryHits(ov))) / length(gr1)
```

```
## [1] 0.3333691
```

and how many percent of promoters have a peak in them

```
length(unique(subjectHits(ov))) / length(promoters)
```

```
## [1] 0.440878
```

My rule of thumb is that any cell type has at most 50% of genes expressed, which fits well with these numbers. We also see that there are many H3K4me3 peaks which do not lie in a genic promoter. This is actually expected.

Now, the overlap is non-empty, but is it interesting or “significant”. Answering this question thoroughly requires thinking deeply about background distributions etc. We will avoid a careful statistical approach to this question, and will instead do a few back-of-the-envelope calculations to get a feel for it.

First we notice that both promoters and peaks are small compared to the size of the human genome:

```
sum(width(reduce(gr1))) / 10^6
```

```
## [1] 11.19044
```

```
sum(width(reduce(promoters))) / 10^6
```

```
## [1] 64.3005
```

(result is in megabases; the human genome is 3000 megabases). Just this fact alone should convince you that the overlap is highly unlikely happen purely by chance. Let us calculate the overlap in megabases:

```
sum(width(intersect(gr1, promoters))) / 10^6
```

```
## [1] 0
```

That’s weird; why is the overlap empty when we know (using `findOverlaps`) that there is plenty of overlap? Turns out the answer is strand. We need to ignore the strand when we do this calculation:

```
sum(width(intersect(gr1, promoters, ignore.strand = TRUE))) / 10^6
```

```
## [1] 3.019608
```

This brings us back to the question of the size of the promoter set above; this is also affected by strand when there is a promoter on each strand which overlaps. Contrast

```
sum(width(reduce(promoters))) / 10^6
```

```
## [1] 64.3005
```

```
sum(width(reduce(promoters, ignore.strand = TRUE))) / 10^6
```

```
## [1] 62.27957
```

Strand can be troublesome!

Let us compute a small 2x2 matrix for which bases are in promoters and/or peaks:

```
prom <- reduce(promoters, ignore.strand = TRUE)
peaks <- reduce(gr1)
both <- intersect(prom, peaks)
only.prom <- setdiff(prom, both)
only.peaks <- setdiff(peaks, both)
overlapMat <- matrix(0,, ncol = 2, nrow = 2)
colnames(overlapMat) <- c("in.peaks", "out.peaks")
rownames(overlapMat) <- c("in.promoters", "out.promoter")
overlapMat[1,1] <- sum(width(both))
overlapMat[1,2] <- sum(width(only.prom))
overlapMat[2,1] <- sum(width(only.peaks))
overlapMat[2,2] <- 3*10^9 - sum(overlapMat)
round(overlapMat / 10^6, 2)
```

```
##           in.peaks out.peaks
## in.promoters   3.02    59.26
## out.promoter   8.17   2929.55
```

Here we have just used the genome size of 3 billion bases. This is not correct. There are many of these bases which have not been sequenced and in addition, there are many bases which cannot be mapped using short reads. This will reduce the genome size.

Nevertheless, let us compute an odds-ratio for this table:

```
oddsRatio <- overlapMat[1,1] * overlapMat[2,2] / (overlapMat[2,1] * over  
oddsRatio
```

```
## [1] 18.26938
```

This odds-ratio shows an enrichment of peaks in promoters. We can get a feel for how much the genome size (which we use incorrectly) affects our result by using a lower bound on the genome size. Let us say 1.5 billion bases:

```
overlapMat[2,2] <- 1.5*10^9  
oddsRatio <- overlapMat[1,1] * overlapMat[2,2] / (overlapMat[2,1] * over  
oddsRatio
```

```
## [1] 9.354362
```

The odds-ratio got smaller, but it is still bigger than 1.

Here we basically says that each base can be assigned to peaks or promoters independently, which is definitely false. So there are many, many reasons why this calculation is not the “right” one. Nevertheless, it appears that we have some enrichment, as expected.

SessionInfo

```
## R version 3.2.1 (2015-06-18)
## Platform: x86_64-apple-darwin13.4.0 (64-bit)
## Running under: OS X 10.10.5 (Yosemite)
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## attached base packages:
## [1] stats4      parallel  methods    stats      graphics  grDevices  utils
## [8] datasets    base
##
## other attached packages:
## [1] AnnotationHub_2.0.3  rtracklayer_1.28.8  GenomicRanges_1.20.5
## [4] GenomeInfoDb_1.4.2  IRanges_2.2.7       S4Vectors_0.6.3
## [7] BiocGenerics_0.14.0  BiocStyle_1.6.0     rmarkdown_0.7
##
## loaded via a namespace (and not attached):
## [1] Rcpp_0.12.0          BiocInstaller_1.18.4
## [3] formatR_1.2          futile.logger_1.4.1
## [5] XVector_0.8.0        bitops_1.0-6
## [7] futile.options_1.0.0 tools_3.2.1
## [9] zlibbioc_1.14.0      digest_0.6.8
## [11] evaluate_0.7.2       RSQLite_1.0.0
## [13] shiny_0.12.2         DBI_0.3.1
## [15] curl_0.9.2           yaml_2.1.13
## [17] stringr_1.0.0        httr_1.0.0
## [19] knitr_1.11           Biostrings_2.36.3
## [21] Biobase_2.28.0       R6_2.1.0
## [23] AnnotationDbi_1.30.1 XML_3.98-1.3
## [25] BiocParallel_1.2.20  lambda.r_1.1.7
## [27] magrittr_1.5         Rsamtools_1.20.4
## [29] htmltools_0.2.6      GenomicAlignments_1.4.1
## [31] mime_0.3             interactiveDisplayBase_1.6.0
## [33] xtable_1.7-4         httpuv_1.3.3
## [35] stringi_0.5-5        RCurl_1.95-4.7
```